

Desktop Client Report

Remote File Folder Client Project

(a simplified Google Drive analogue)

Yana Utochkina

Stage: Desktop (macOS/SwiftUI) Client – Design, Usage, and
Testing
September 30, 2026

Abstract

This report documents the **macOS desktop client** of the **Remote File Folder Client** project (desktop/Sources/FileSpaceDesktop/), a simplified analogue of Google Drive with web and desktop clients, a Go backend, PostgreSQL, and local-folder synchronization. It describes the client's internal structure, walks through its use cases with screenshots, and documents its unit test coverage, including its sorting and filtering behavior (sort by *Edited by*, filter by *.cs/.jpg*).

Contents

1	Introduction	2
2	Desktop Client Architecture	2
2.1	Internal Structure	2
2.2	Synchronization	2
3	Use Cases and Screenshots	3
3.1	Register and Authenticate	3
3.2	Browse Files: Attributes, Columns, Sorting, Filtering	4
3.3	View File Content	6
3.4	Upload, Download, Delete	7
3.5	Synchronize a Local Folder	8

4	Unit Testing	9
4.1	Test Setup	9
4.2	Coverage by Area	10
4.3	Sorting and Filtering Coverage	10
5	Conclusions	11

1. Introduction

The desktop client is one of two front ends (the other being the React web client) to a single Go API and PostgreSQL database, self-hosted on a home Ubuntu server behind a Cloudflare Tunnel. Every use case the system supports through either client – registration, authentication, browsing files with sortable/filterable attributes, viewing `.java/.png` content, uploading, downloading, deleting – is available from the desktop client; it is additionally the *only* client that implements continuous background folder synchronization, since browsers cannot watch a local folder persistently.

2. Desktop Client Architecture

2.1 Internal Structure

The desktop client's source folder, `desktop/Sources/FileSpaceDesktop/`, groups its code as follows: SwiftUI views are kept separate from the sync engine (`DirectoryWatcher`, `SyncViewModel`, and `Manifest.swift`'s `SyncState/FolderScanner`) and from the shared `APIClient/KeychainStore` networking layer:

- **Views:** `LoginView`, `MainView` (file table, sorting/filtering/column visibility), `FilePreviewView(.java/.png)`, `SyncPanelView` and `SyncConflictsSectionView` (folder sync controls and conflict resolution UI), `ContentView` (top-level navigation between logged-out/logged-in state);
- **View models:** `FilesViewModel` (file list, sort/filter, upload/download/delete), `SyncViewModel` (reconcile loop, conflict detection/resolution), `AppState` (session/auth state);
- **Networking & storage:** `APIClient` (URLSession-based REST client, behind the `APIClientProtocol` seam used for testing), `KeychainStore` (token storage via the macOS Keychain), `Models.swift` (Codable DTOs matching the Go API's JSON shapes);
- **Sync engine:** `DirectoryWatcher` (`FSEvents` wrapper), `Manifest.swift` (`FolderScanner`, `SyncState`, `SyncedFileEntry`).

The desktop client talks to the Go API server only over HTTPS through the public Cloudflare Tunnel hostname – the same endpoint the web client's browser JavaScript uses – passing through the server's `Handler` → `Service` → `Repository` layers and, for file content, the `FileStorage` interface.

2.2 Synchronization

Synchronization is the desktop client's most involved feature and the one with no web-client equivalent. On each trigger – an `FSEvents` change notification from `DirectoryWatcher`, or a periodic poll – `SyncViewModel` calls `POST /api/sync/diff` with its `lastSyncedVersion` and a manifest

(name, size, sha256) computed by FolderScanner; the server replies with download/delete_local/conflict actions plus a newVersion. The client applies those actions, then independently scans its local folder and uploads, re-uploads, or deletes files based on its own persisted SyncState.

Conflict resolution

A conflict action means the same file changed on both sides since the last sync: the server compares the client's submitted manifest (name, size, sha256) against its own record and returns a conflict action carrying the remote side's size, modification time, and last editor whenever the two hashes genuinely disagree. On the desktop client, SyncViewModel records such actions in a published conflicts list rather than resolving them automatically, and SyncConflictsSectionView (part of the existing sync panel) lets the user choose **Keep Local** (re-upload, overwriting the remote copy), **Keep Remote** (overwrite the local file), or **Keep Both** (keep the local file under a renamed copy and adopt the remote version).

3. Use Cases and Screenshots

This section walks through the desktop client's use cases, each with a screenshot of the running application.

3.1 Register and Authenticate

Register (fig. 1) creates a new account by username/password; *Authenticate* (fig. 2) is required by every other file operation, since the workspace is only reachable after a successful login, after which APIClient stores the returned access/refresh JWT pair in the macOS Keychain via KeychainStore.

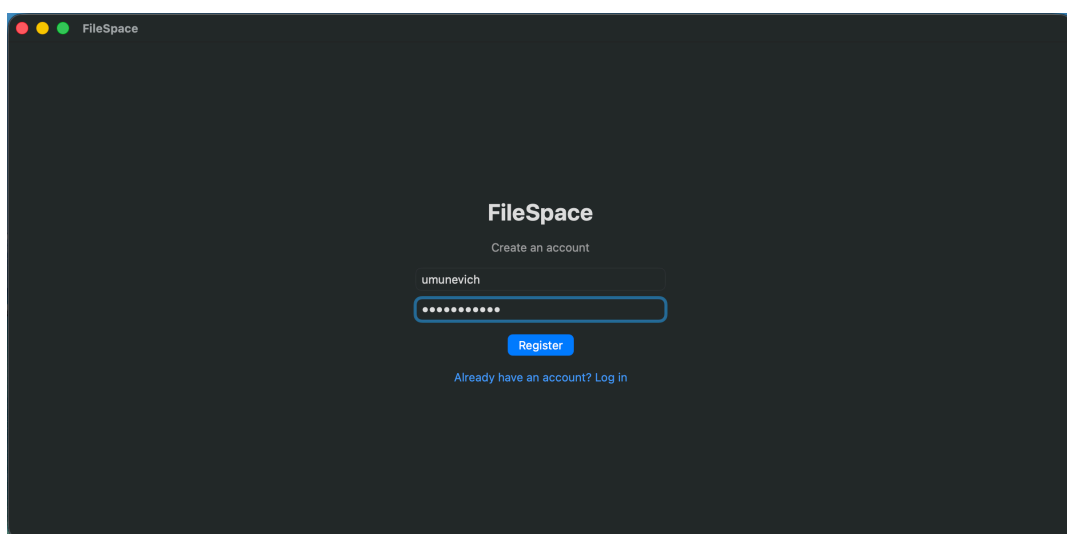


Figure 1: Registration screen

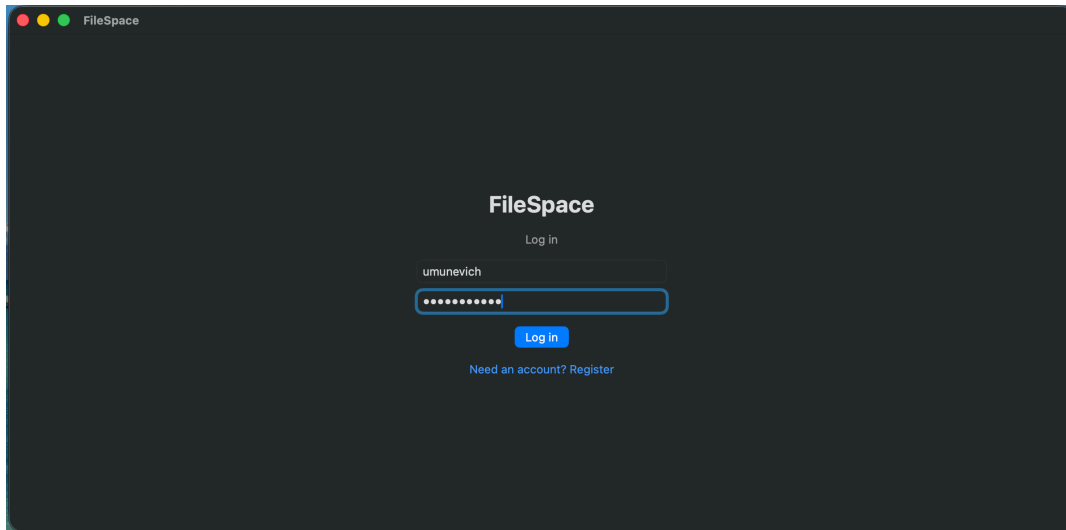


Figure 2: Login screen

3.2 Browse Files: Attributes, Columns, Sorting, Filtering

Figure 3 shows the main file table with every attribute column visible (name, created, modified, uploaded by, edited by); the name column cannot be hidden, while the others toggle through the “Columns” menu (fig. 4).

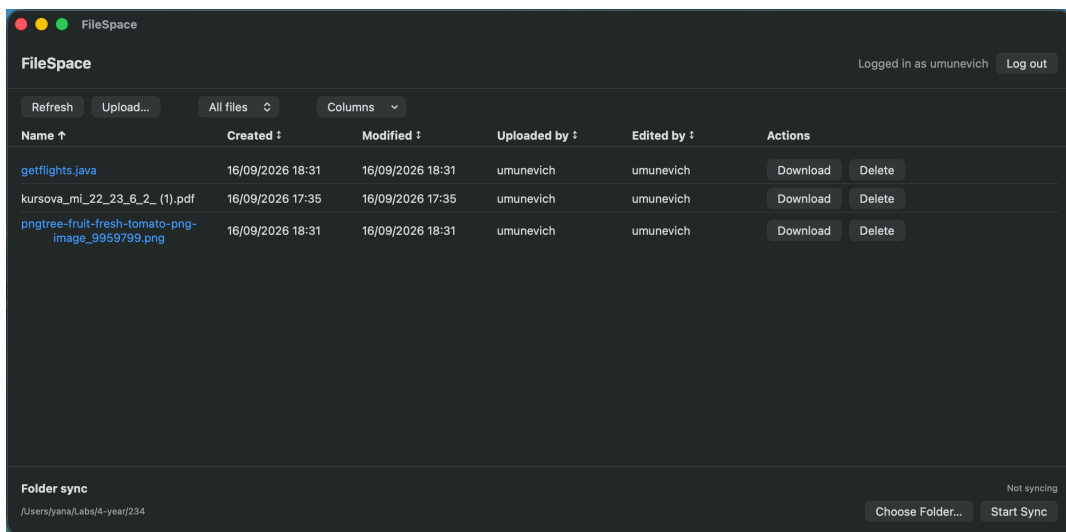


Figure 3: Main file list with all attribute columns visible

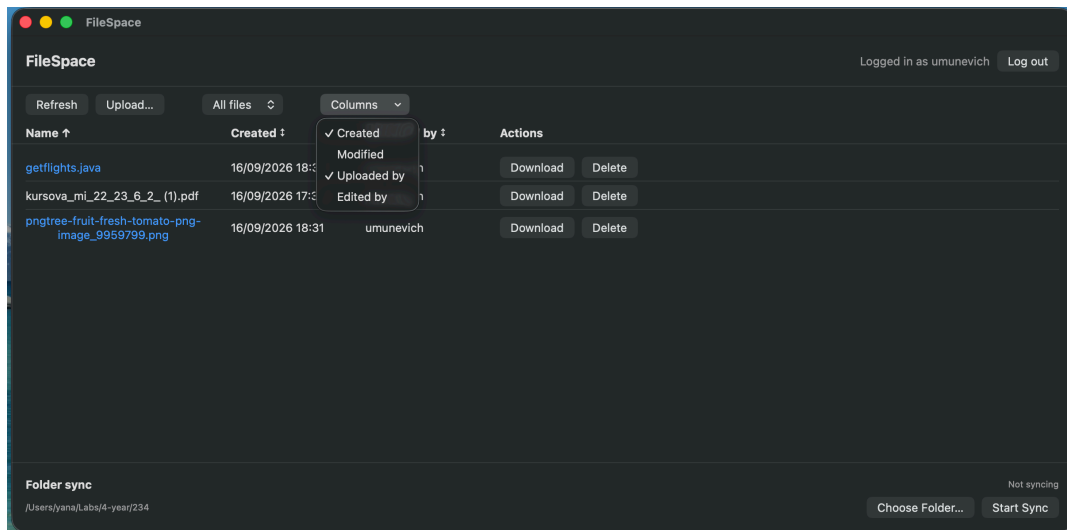


Figure 4: Columns menu: toggling a non-name column

Sorting: the file list sorts by *Edited by*, ascending or descending, by clicking that column header, which cycles `FilesViewModel.sortDirection` through none → ascending → descending → none (fig. 5).

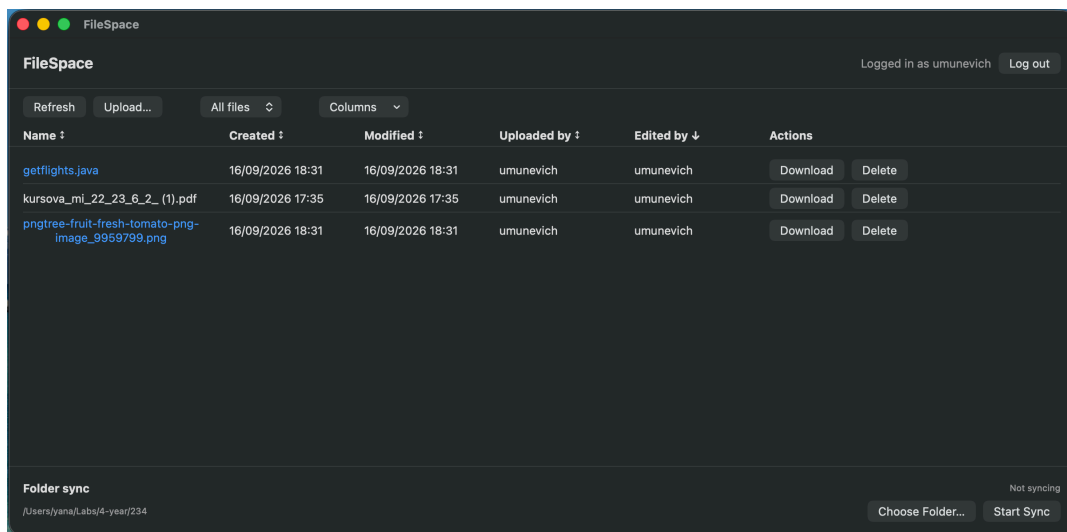


Figure 5: File list sorted by “Edited by”

Filtering: the extension dropdown, populated from the extensions actually present among the user’s files, filters the list down to just `.cs` or just `.jpg` (or back to “All”) via `FilesViewModel.extensionFilter` (fig. 6).

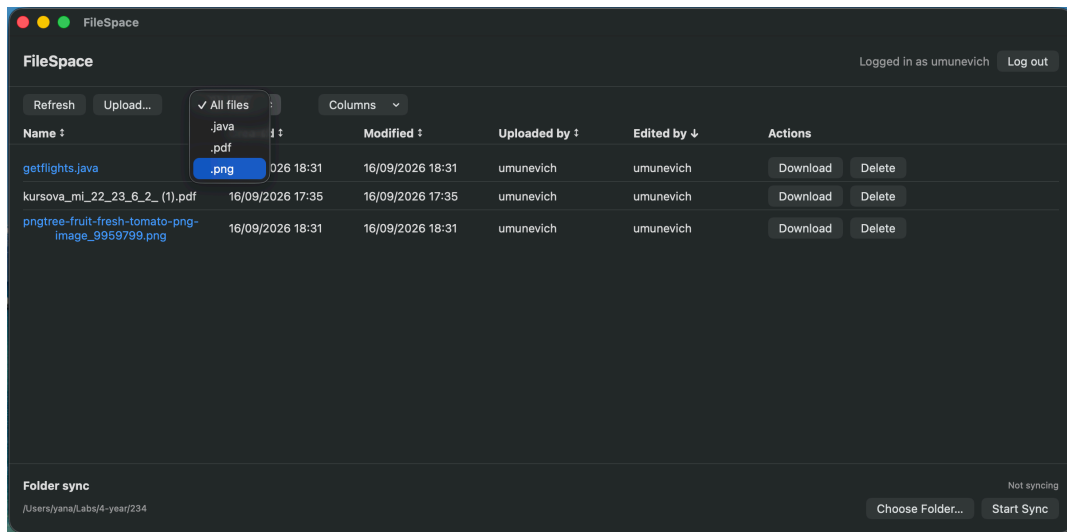


Figure 6: Extension filter applied (.cs or .jpg)

3.3 View File Content

Clicking a file previews its content if the extension is .java (rendered as text, fig. 7) or .png (rendered as an image, fig. 8); every other extension is browsed/downloaded but not previewed.

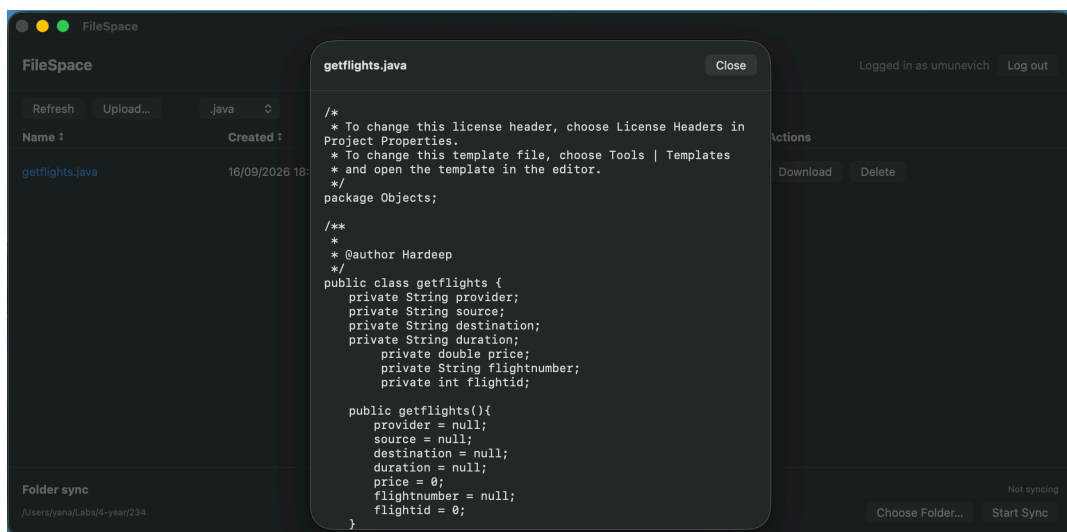


Figure 7: .java file previewed as text

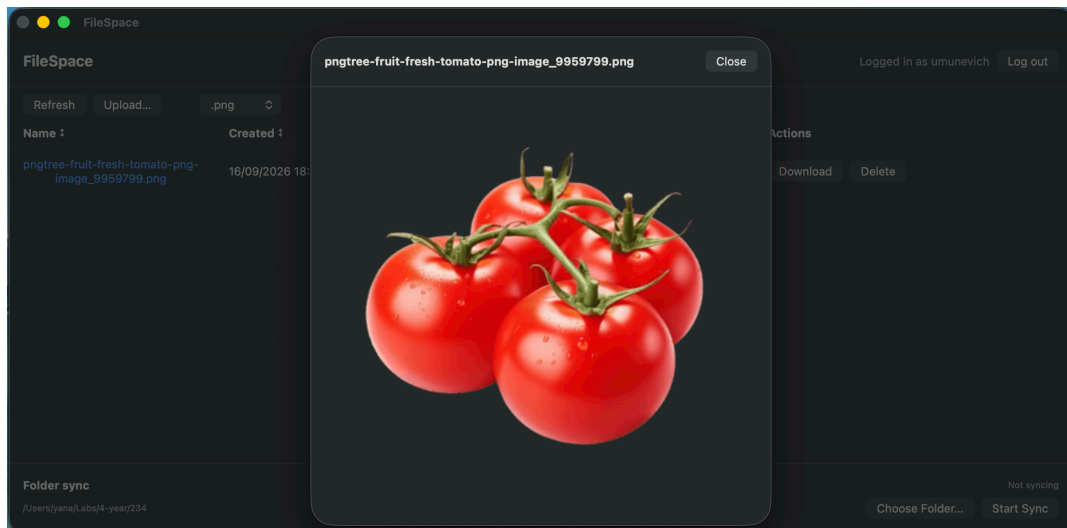


Figure 8: .png file previewed as an image

3.4 Upload, Download, Delete

Figure 9 shows an upload in progress – large files are split into chunks so no single HTTP request exceeds the Cloudflare Tunnel free-tier body-size limit, with uploadProgress reflecting bytes sent per chunk. Figures 10 and 11 show downloading a file to the user’s Downloads folder and deleting one from the remote workspace.

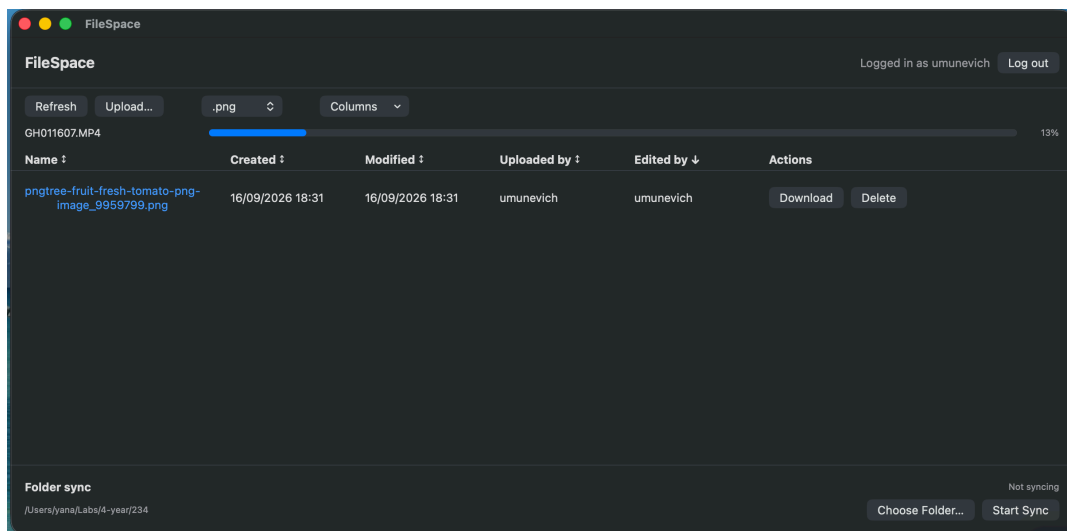


Figure 9: File upload in progress

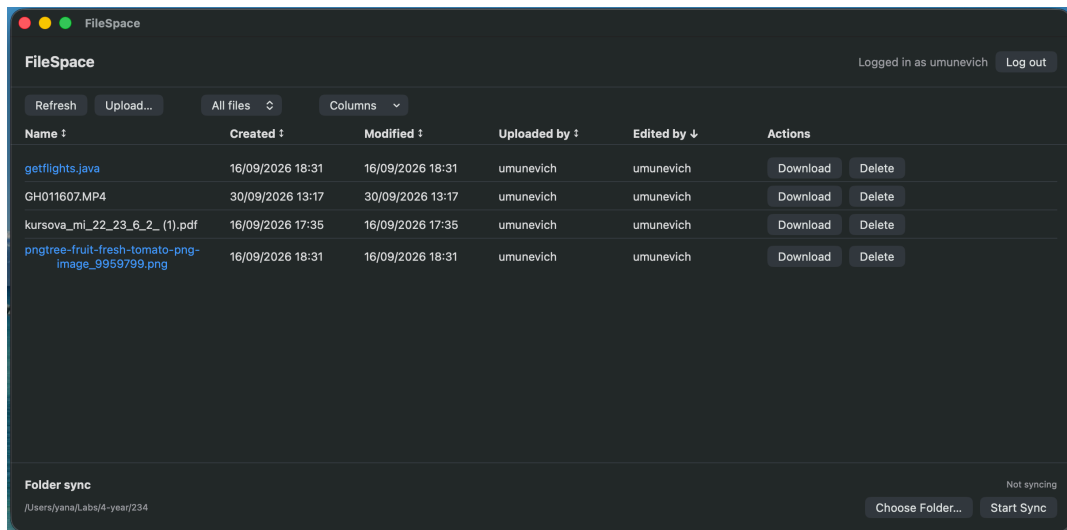


Figure 10: File download

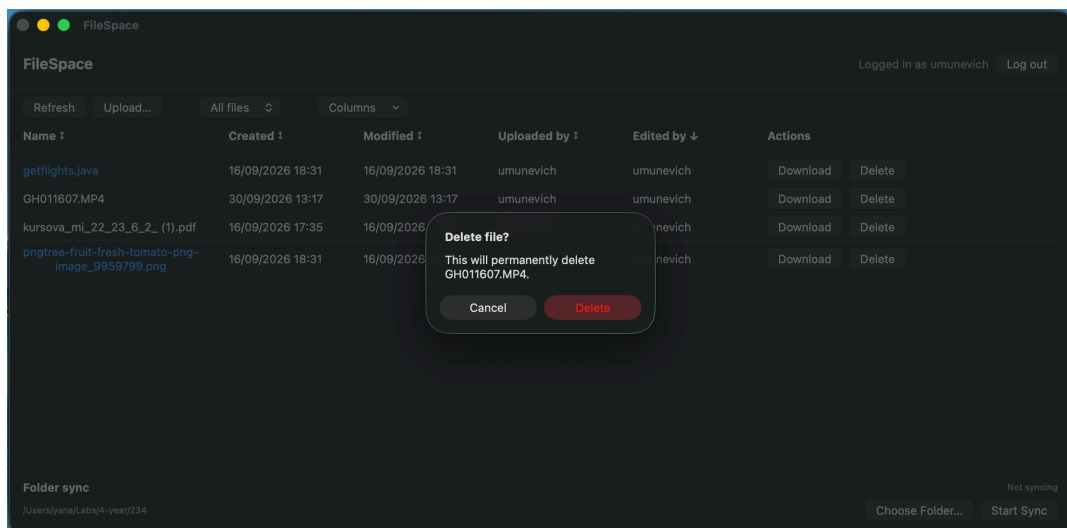


Figure 11: File deletion

3.5 Synchronize a Local Folder

Figure 12 shows the sync panel with a folder chosen and synchronization running: DirectoryWatcher raises FSEvents-based change notifications, and SyncViewModel also polls periodically, running the reconcile cycle described in §2.2. Figure 13 shows the same panel when the server reports a genuine conflict: the file is listed with its local/remote size and editor, alongside the Keep Local / Keep Remote / Keep Both actions.

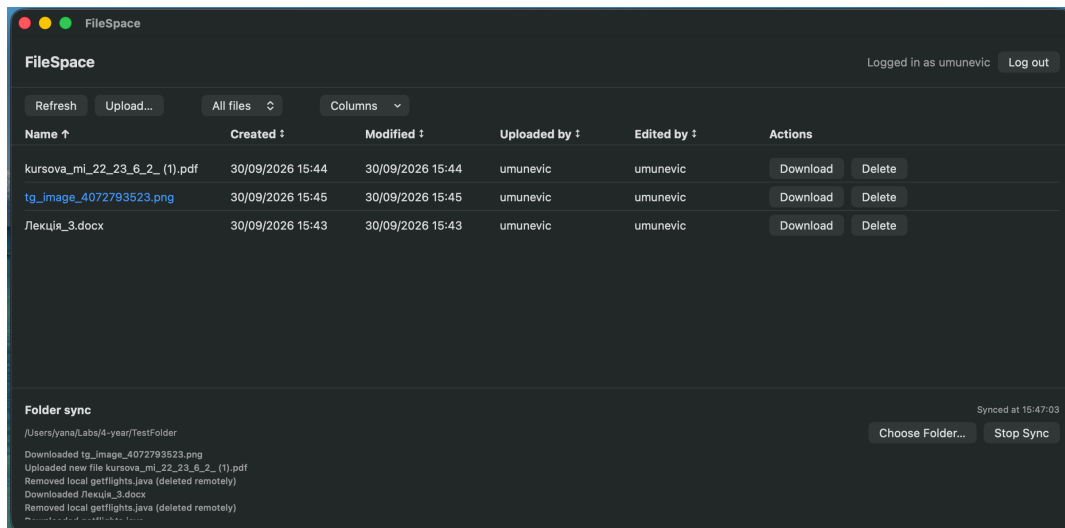


Figure 12: Folder sync panel, actively syncing

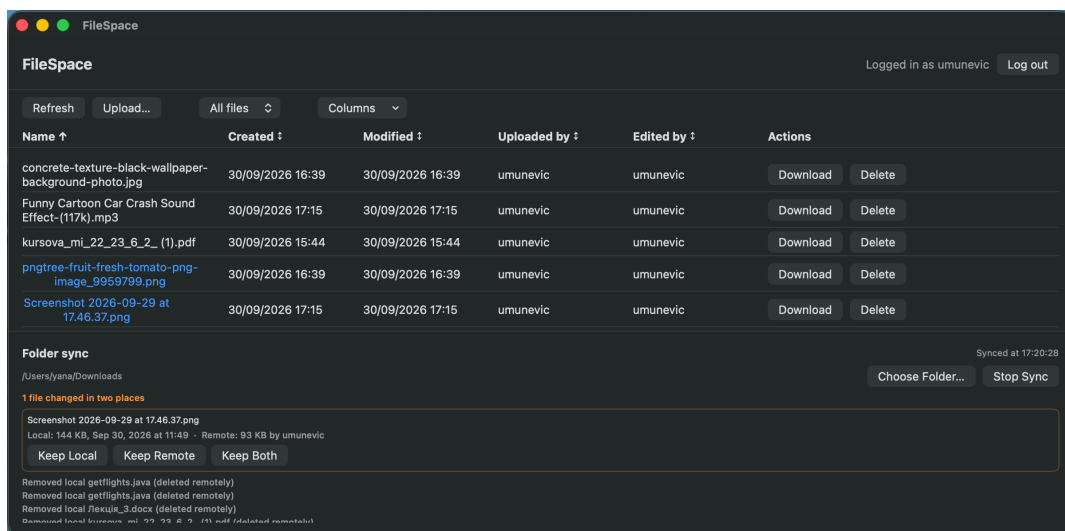


Figure 13: Folder sync panel showing a detected conflict and its resolution options

4. Unit Testing

4.1 Test Setup

The desktop client's test target, `FileSpaceDesktopTests`, uses `XCTest` (run via `swift test` from `desktop/`, or `make desktop-test`) rather than the newer `swift-testing` module, since this environment ships only the Xcode Command Line Tools, which do not include the Testing module. Networked or OS-dependent collaborators are substituted with in-memory fakes so the suite runs offline and deterministically:

- `SyncViewModel` and `FilesViewModel` depend on `APIClientProtocol` rather than the concrete `APIClient`, so tests inject a no-network

- FakeAPIClient in place of URLSession;
- FolderScanner and sync-state tests use real temporary directories on disk rather than mocking the filesystem, since the code under test is filesystem interaction;
- KeychainStoreTests exercise the real macOS Keychain (there is no in-memory substitute for it) using dedicated keys, cleaned up in tearDown.

4.2 Coverage by Area

Table 1 summarizes the test suite.

Table 1: Unit test suites in desktop/Tests/FileSpaceDesktopTests/

Test class	Tests	What it covers
FileRecordDecoding-Tests	2	FileRecord decodes the backend's exact JSON shape, including the extension → extensionName key remap
SyncStateTests	1	SyncState/SyncedFileEntry round-trip through JSON encoding
FolderScannerTests	4	Local folder scanning: hash computation, skipping hidden files/subdirectories, and correctly distinguishing “can’t be read” from “doesn’t exist”
PreviewSupportTests	1	Only .java/.png are treated as previewable
KeychainStoreTests	5	Token set/read/overwrite/delete against the real macOS Keychain, with key isolation
APIClientTests	6	Request/response decoding, chunked upload ordering and progress, single-shot vs. chunked upload selection, and 401-refresh-then-retry behavior
SyncViewModelTests	11	The reconcile loop: new/changed/deleted-local file handling, manifest submission, and conflict detection/recording plus all three resolutions (Keep Local/Remote/Both)
FilesViewModelTests	7	Sorting by <i>Edited by</i> (ascending, descending, case-insensitivity) and filtering by extension (.cs, .jpg, no filter), including sort+filter composed together
Total	37	

4.3 Sorting and Filtering Coverage

The file list sorts by “**Edited by**” (ascending and descending) and filters to **all files**, only .cs files, or only .jpg files. This logic lives in FilesViewModel.displayedFiles, toggleSort(_:), and extensionFilter, and is exercised by FilesViewModelTests.swift:

```
testSortByEditedByAscending
testSortByEditedByDescending
```

```
testSortByEditedByIsCaseInsensitive
testNoFilterShowsAllExtensions
testFilterByCsExtensionShowsOnlyCsFiles
testFilterByJpgExtensionShowsOnlyJpgFiles
testFilterAndSortByEditedByComposeTogether
```

FilesViewModel depends on APIClientProtocol rather than the concrete, network-backed APIClient (the same seam SyncViewModel uses), which is what lets these tests inject a fake client and run offline.

```
Test Suite 'All tests' passed at 2026-09-30 10:53:05.
  Executed 37 tests, with 0 failures (0 unexpected)
  in 0.190 (0.197) seconds
```

5. Conclusions

The desktop client implements every use case the project requires that a native macOS app can meaningfully add over the web client: full file browsing with sorting by “Edited by” and filtering by .cs/.jpg, content preview for .java/.png, upload/download/delete, and – uniquely among the two clients – continuous background folder synchronization, including manifest-based conflict detection and a three-way (Keep Local/Remote/Both) resolution UI. On the testing side, the client’s core collaborators (networking, folder scanning, sync state, Keychain access, the sync reconcile loop, and file-list sorting/filtering) all carry unit test coverage. All 37 tests in the suite pass.